

CarryBee Courier API — Integration Guide

Reference doc distilled from merchant.carrybee.com (Quick Setup, API Credentials, API Reference, Webhook Integration, Webhook Events, API Documentation pages) on 2026-09-09. Written so it can be dropped into a repo and fed to Claude Code as authoritative context for building the integration.

Stack assumption: **NestJS (TypeScript) backend**, Prisma/Mongoose for persistence, BullMQ for async/queue work, Redis available. Adjust section 9 if the target service differs.

1. Overview

CarryBee is a Bangladesh courier/delivery API. It lets a merchant:

- Look up delivery **locations** (city → zone → area).
- Register **pickup stores**.
- Create, cancel and track **orders** (shipments), including bulk creation.
- Create **reverse pickups** (returns) and **exchange orders**.
- Receive near real-time status updates via **webhooks** instead of polling.

All requests are JSON over HTTPS. All list/detail responses share a consistent envelope; errors share a consistent shape (see §5).

2. Environments

Two fully isolated environments, each with its own credentials and base URL. Build and test against Sandbox first; going live is just swapping the base URL + credentials, no code changes.

Environment	Base URL
Sandbox	https://sandbox.carrybee.com
Production	https://developers.carrybee.com

3. Authentication

Every request (except none — all endpoints require this) must include three headers, issued per business **and** per environment:

Client-ID: <your-client-id>

Client-Secret: <your-client-secret>

Client-Context: <your-client-context>

Content-Type: application/json # required for POST/PUT

Get these from the CarryBee merchant dashboard → **API Credentials** page (separate Sandbox and Production sets). **Never expose Client-Secret in front-end code** — all calls must be made server-side.

Getting credentials (one-time setup)

1. Log into the CarryBee merchant dashboard → **API Credentials**.
 2. Copy the Sandbox **Client-ID**, **Client-Secret**, **Client-Context**.
 3. Store them as environment variables (see §10) — do not commit them to git or paste them into docs/PRs.
 4. Repeat for Production once you're ready to go live.
-

4. Conventions

Common field values/units used across the API:

Field	Meaning
delivery_type	1 = Normal, 2 = Express
product_type	1 = Parcel, 2 = Book, 3 = Document
item_weight	grams, range 1 – 25000
item_quantity	range 1 – 200
collectable_amount	Taka (COD amount), range 0 – 100000

Field	Meaning
timestamps	ISO 8601 with timezone (e.g. 2025-07-30T10:11:12+00:00)

5. Responses & errors

Successful responses return JSON wrapped like:

```
{  
  
  "error": false,  
  
  "message": "Order created successfully",  
  
  "data": {  
  
    "order": { "...": "..." }  
  
  }  
}
```

Errors share a consistent shape:

Standard error

```
{  
  
  "error": true,  
  
  "message": "descriptive error message"  
}
```

Validation error (422)

```
{  
  
  "error": true,
```

```
"message": "Validation error",

"causes": {

  "field_name": [

    { "type": "validation_type", "attribute": {} }

  ]

}

}
```

Status	Meaning
200	Success
201	Created
202	Accepted (queued / async — e.g. bulk orders, cancellation)
404	Not found
422	Validation error (see causes)
4xx / 5xx	Client / server error

6. API Reference

All paths are relative to the environment base URL (§2). All require the three auth headers from §3.

6.1 Location

[GET /api/v2/cities](#)

Retrieves available cities for delivery operations.

Response:

```
{
  "error": false,
  "message": "City list fetched successfully",
  "data": { "cities": [{ "id": 1, "name": "Bagerhat" }, { "id": 14, "name": "Dhaka" }] }
}
```

GET /api/v2/cities/{city_id}/zones

Fetches zones within a specified city. | Param | Type | Required | |---|---|---| | **city_id** (path) | integer | ✓ |

GET /api/v2/cities/{city_id}/zones/{zone_id}/areas

Retrieves areas within a zone. | Param | Type | Required | |---|---|---| | **city_id** (path) | integer | ✓ | | **zone_id** (path) | integer | ✓ |

GET /api/v2/area-suggestion

Searches for areas, zones, or cities by partial text match. | Param | Type | Required | |---|---|---| | **search** (query) | string | ✓ · min 3 chars |

POST /api/v2/address-details

Resolves a free-text address string to city/zone identifiers. | Param | Type | Required | |---|---|---| | **query** (body) | string | ✓ · min 10 chars |

6.2 Stores

POST /api/v2/stores

Creates a new merchant (pickup) store. Parcels are collected from an approved store — create one before placing orders.

Param	Type	Required	Notes
name	string	✓	3–30 chars
contact_person_name	string	✓	3–30 chars

Param	Type	Required	Notes
contact_person_number	string	✓	valid phone
contact_person_secondary_number	string	optional	phone
address	string	✓	3–100 chars
city_id	integer	✓	
zone_id	integer	✓	
area_id	integer	✓	
lat	number	optional	
lng	number	optional	

curl --request POST \

--url https://sandbox.carrybee.com/api/v2/stores \

--header 'Client-ID: <your-client-id>' \

--header 'Client-Secret: <your-client-secret>' \

--header 'Client-Context: <your-client-context>' \

--header 'Content-Type: application/json' \

--data '{

"name": "Main Warehouse",

"contact_person_name": "Rahim Uddin",

"contact_person_number": "01700000000",

"address": "House 1, Road 2, Gulshan",

"city_id": 1,

```
"zone_id": 10,  
  
"area_id": 100  
  
}'
```

Response: 201 Created

GET /api/v2/stores

Lists all stores for the authenticated merchant. No extra params.

6.3 Orders

POST /api/v2/orders

Creates a single shipment order.

Param	Type	Required	Notes
store_id	string	✓	from Create a store
merchant_order_id	string	optional	< 50 chars, your own order/tracking id
delivery_type	integer	✓	1=Normal, 2=Express
product_type	integer	✓	1=Parcel, 2=Book, 3=Document
recipient_phone	string	✓	valid phone
recipient_secendary_phone	string	optional	phone (<i>note: typo in API's own field name — "secendary"</i>)
recipient_name	string	✓	2–99 chars
recipient_addresses	string	✓	10–200 chars
city_id	integer	✓	

Param	Type	Required	Notes
zone_id	integer	✓	
area_id	integer	optional	
special_instruction	string	optional	< 255 chars
product_description	string	optional	< 255 chars
item_weight	integer	✓	grams, 1–25000
item_quantity	integer	optional	1–200
collectable_amount	integer	optional	Taka (COD), 0–100000
is_closed_box	boolean	optional	default false
is_exchange	boolean	optional	default false

```

curl --request POST \

--url https://sandbox.carrybee.com/api/v2/orders \

--header 'Client-ID: <your-client-id>' \

--header 'Client-Secret: <your-client-secret>' \

--header 'Client-Context: <your-client-context>' \

--header 'Content-Type: application/json' \

--data '{

  "store_id": "<store-id>",

  "merchant_order_id": "order-1234",

  "delivery_type": 1,

  "product_type": 1,

```

```
"recipient_name": "Karim",  
  
"recipient_phone": "01800000000",  
  
"recipient_address": "House 5, Road 3, Banani",  
  
"city_id": 1,  
  
"zone_id": 10,  
  
"item_weight": 500,  
  
"collectable_amount": 1500  
  
'
```

Response 201 Created:

```
{  
  
  "error": false,  
  
  "message": "Order created successfully",  
  
  "data": {  
  
    "order": {  
  
      "consignment_id": "FX1212124433",  
  
      "store_id": "<store-id>",  
  
      "merchant_order_id": "order-1234",  
  
      "collectable_amount": "1500",  
  
      "cod_fee": 15,  
  
      "delivery_fee": "83.46"  
  
    }  
  
  }  
}
```

```
}  
  
}
```

`consignment_id` is the key you use for tracking, cancelling, reverse-pickup-from-order and exchange calls.

POST `/api/v2/orders-bulk`

Creates multiple orders in one batch (array of the same order object as above). A webhook is sent per resulting order; validation errors are keyed by array position (e.g. "`0.city_id`").

```
[  
  
  {  
  
    "store_id": "string",  
  
    "merchant_order_id": "order-1",  
  
    "delivery_type": 1,  
  
    "product_type": 1,  
  
    "recipient_phone": "01800000000",  
  
    "recipient_name": "Karim",  
  
    "recipient_address": "House 5, Road 3, Banani",  
  
    "city_id": 1,  
  
    "zone_id": 10,  
  
    "item_weight": 500,  
  
    "collectable_amount": 1500  
  
  }  
  
]
```

Response: `202 Accepted`

GET /api/v2/orders/{consignment_id}/details

Retrieves detailed order status/info. Accepts either the CarryBee `consignment_id` or your own `merchant_order_id`. | Param | Type | Required | |---|---|---| | `consignment_id` (path) | string | ✓ · min 3 chars |

POST /api/v2/orders/{consignment_id}/cancel

Cancels an existing order. | Param | Type | Required | |---|---|---| | `consignment_id` (path) | string | ✓ | | `cancellation_reason` (body) | string | ✓ · < 200 chars |

Response: 202 Accepted

6.4 Reverse Pickup

POST /api/v2/orders/reverse-pickup

Creates a standalone reverse pickup (return shipment) request.

Param	Type	Required	Notes
<code>store_id</code>	string	✓	
<code>merchant_order_id</code>	string	optional	
<code>product_type</code>	integer	✓	1=Parcel, 2=Book, 3=Document
<code>customer_phone</code>	string	✓	valid phone
<code>customer_secondary_phone</code>	string	optional	
<code>customer_name</code>	string	✓	2–99 chars
<code>customer_address</code>	string	✓	10–200 chars
<code>city_id</code>	integer	✓	
<code>zone_id</code>	integer	✓	
<code>area_id</code>	integer	optional	

Param	Type	Required	Notes
item_weight	integer	✓	grams, 1–25000
item_quantity	integer	optional	1–200
product_value	integer	optional	1–100000
product_description	string	optional	< 255 chars
special_instruction	string	optional	< 255 chars

Response: 201 Created

POST /api/v2/orders/{consignment_id}/reverse-pickup

Creates a reverse pickup **from an existing order**. That order must be **delivered** or **partially delivered**. Any field you omit falls back to the original order's value.

Param	Type	Required	Notes
consignment_id (path)	string	✓	the original order to reverse
store_id	string	optional	defaults to reference order
merchant_order_id	string	optional	< 50 chars
customer_phone	string	optional	defaults to reference order
customer_secondary_phone	string	optional	
customer_name	string	optional	2–99 chars
customer_address	string	optional	10–200 chars
city_id	integer	optional	defaults to reference order

Param	Type	Required	Notes
zone_id	integer	optional	defaults to reference order
area_id	integer	optional	
item_weight	integer	optional	grams, 1–25000
item_quantity	integer	optional	1–200
product_value	integer	optional	1–100000
product_description	string	optional	< 255 chars
special_instruction	string	optional	< 255 chars

Response: 201 Created

6.5 Exchange

POST /api/v2/orders/{consignment_id}/exchange

Creates an exchange order from a **delivered** shipment. The new consignment id is prefixed with EX.

Param	Type	Required	Notes
consignment_id (path)	string	✓	the delivered order to exchange
merchant_order_id	string	optional	< 50 chars
collectable_amount	integer	optional	Taka, 0–100000
item_quantity	integer	optional	default 1
item_weight	integer	optional	grams, 1–25000

Param	Type	Required	Notes
product_description	string	optional	< 255 chars
special_instruction	string	optional	< 255 chars

Response: 201 Created

7. Webhooks

Instead of polling `GET /orders/{id}/details`, register a webhook URL so CarryBee pushes a POST to you for every status change, near real-time.

7.1 Setup — the integration handshake

Set the webhook URL on the merchant dashboard → **Webhook Integration** page. CarryBee immediately sends a verification POST:

```
{ "event": "webhook.integration" }
```

Your endpoint **must**, for this specific request:

- Be publicly reachable, resolve within ≤ 3 redirections.
- Use a valid SSL certificate if HTTPS, and respond within the timeout.
- Return HTTP **202**.
- Echo the header `X-CB-Webhook-Integration-Header` back in the response, with **exactly** the secret value CarryBee sent you for that integration (shown on the Webhook Integration page when you set it up).

If this handshake fails, the webhook won't be considered active.

7.2 Verifying real event requests

Every subsequent webhook POST includes:

X-Carrybee-Webhook-Signature: "{{WEBHOOK_SIGNATURE}}"

Validate this signature header server-side before trusting the payload, so you know the request actually came from CarryBee and wasn't spoofed. (Compare against the webhook secret

configured for your integration — verify this exact comparison mechanism against the dashboard's webhook secret field before relying on it in production, the docs describe the header but not the hashing algorithm in detail.)

7.3 Common payload envelope

Almost all events share this base shape, with a few adding extra fields:

```
{  
  
  "event": "order.<status>",  
  
  "store_id": "a1b2c3d4",  
  
  "consignment_id": "FX1212124433",  
  
  "merchant_order_id": "order-1234",  
  
  "timestampz": "2025-07-30T10:11:12+00:00"  
}
```

7.4 Event catalog

Event name	Event code	Extra payload fields beyond the base envelope
Order Created	order.created	collectable_amount, cod_fee, delivery_fee
Order Create Failed	order.create-failed	(base only)
Order Updated	order.updated	collectable_amount, cod_fee, delivery_fee
Pickup Requested	order.pickup-requested	(base only)
Assigned For Pickup	order.assigned-for-pickup	(base only)
Picked	order.picked	(base only)
Pickup Failed	order.pickup-failed	(base only)
Pickup Cancelled	order.pickup-cancelled	(base only)

Event name	Event code	Extra payload fields beyond the base envelope
At The Sorting Hub	<code>order.at-the-sorting-hub</code>	(base only)
On The Way To Central Warehouse	<code>order.on-the-way-to-central-warehouse</code>	(base only)
At Central Warehouse	<code>order.at-central-warehouse</code>	(base only)
In Transit	<code>order.in-transit</code>	(base only)
Received At Last Mile Hub	<code>order.received-at-last-mile-hub</code>	(base only)
Assigned For Delivery	<code>order.assigned-for-delivery</code>	(base only)
Delivery On Hold	<code>order.delivery-on-hold</code>	(base only)
Delivered	<code>order.delivered</code>	<code>collected_amount, attempt, remarks</code>
Partial Delivery	<code>order.partial-delivery</code>	<code>collected_amount, attempt, reason, remarks</code>
Delivery Failed	<code>order.delivery-failed</code>	<code>attempt, reason, remarks</code>
Returned	<code>order.returned</code>	<code>reason, remarks</code>
Paid Return	<code>order.paid-return</code>	<code>collected_amount, attempt, reason, remarks</code>
Exchange	<code>order.exchange</code>	<code>collected_amount, reason, remarks</code>
Paid	<code>order.paid</code>	(base only, confirm exact shape in sandbox)
Returned At Sorting	<code>order.returned-at-sorting</code>	(base only)
Returned In Transit	<code>order.returned-in-transit</code>	(base only)

Event name	Event code	Extra payload fields beyond the base envelope
Returned To Merchant	<code>order.returned-to-merchant</code>	(base only)

`reason` / `remarks` are documented as "may or may not contain data" — treat as nullable strings.

Example — `order.delivered`:

```
{
  "event": "order.delivered",
  "store_id": "a1b2c3d4",
  "consignment_id": "FX1212124433",
  "merchant_order_id": "order-1234",
  "timestampz": "2025-07-30T10:11:12+00:00",
  "collected_amount": 60,
  "attempt": 1,
  "remarks": "This field may or may not contain data"
}
```

Example — `order.delivery-failed`:

```
{
  "event": "order.delivery-failed",
  "store_id": "a1b2c3d4",
  "consignment_id": "FX1212124433",
  "merchant_order_id": "order-1234",
```

```
"timestampz": "2025-07-30T10:11:12+00:00",

"attempt": 1,

"reason": "This field may or may not contain data",

"remarks": "This field may or may not contain data"

}
```

You can pick a subset of events relevant to your business on the Webhook Integration page, and fire test events from the Webhook Events page in Sandbox.

8. Quick Setup checklist (sandbox → production)

1. Get Sandbox API credentials (Client-ID / Client-Secret / Client-Context).
 2. Send the three auth headers + `Content-Type: application/json` on every request.
 3. Create a pickup store (`POST /api/v2/stores`), keep its `store_id`.
 4. Place a test order (`POST /api/v2/orders`), keep the returned `consignment_id`.
 5. Track it (`GET /api/v2/orders/{consignment_id}/details`).
 6. Register a webhook URL and complete the integration handshake (§7.1).
 7. Subscribe to the events your app needs (§7.4) and fire test events.
 8. Swap to Production credentials + <https://developers.carrybee.com> base URL.
No code changes needed — only config.
-

9. Suggested integration shape (NestJS + TypeScript)

This is a suggested module layout, not something CarryBee prescribes — adapt to the existing project structure.

src/

 courier/

 carrybee/

 carrybee.module.ts

carrybee.config.ts # reads env vars, picks base URL by environment

carrybee.client.ts # thin HTTP client: sets auth headers, base URL, error mapping

carrybee.service.ts # createOrder, createBulkOrders, getOrderDetails, cancelOrder,
 # createReversePickup, createReversePickupFromOrder, createExchange,
 # listCities, listZones, listAreas, searchAreas, resolveAddress,
 # createStore, listStores

dto/

create-order.dto.ts

create-store.dto.ts

reverse-pickup.dto.ts

exchange-order.dto.ts

webhook/

carrybee-webhook.controller.ts # POST endpoint CarryBee calls

carrybee-webhook.guard.ts # verifies X-Carrybee-Webhook-Signature /
 # echoes X-CB-Webhook-Integration-Header for the
 # one-time "webhook.integration" handshake

carrybee-webhook.types.ts # discriminated union keyed by `event`

carrybee-webhook.processor.ts # BullMQ processor: idempotent handling per event

Points worth calling out for whoever (or whichever agent) implements this:

- **Config:** `CARRYBEE_ENV=sandbox|production` selects base URL; separate env var pairs for sandbox/production Client-ID/Secret/Context so switching environments never touches code (see §10).

- **Client-Secret never leaves the server.** Don't return it from any API you expose to the frontend; don't log it.
 - **Idempotency:** webhook events can, in principle, be retried/duplicated by any provider — dedupe on (`consignment_id`, `event`, `timestampz`) before processing, e.g. a unique index or a Redis SETNX with short TTL.
 - **Webhook handler should ack fast:** return 202 quickly, push the payload onto a BullMQ queue, and process (update order status in DB, notify user, etc.) in a worker — don't do heavy work synchronously in the controller since CarryBee's retry/timeout behavior for slow endpoints isn't documented.
 - **The one-time `webhook.integration` event is special** — it must get a **202** with the `X-CB-Webhook-Integration-Header` echoed back, not routed through the normal event-processing path.
 - **Validation:** mirror the field constraints from §6 (phone format, string lengths, weight/amount ranges) in your DTOs with `class-validator` so bad requests fail fast locally instead of round-tripping to CarryBee for a 422.
 - **`recipient_secendary_phone`** — note the typo is in CarryBee's own field name (not a mistake in this doc); your DTO's wire field must match theirs exactly even though it reads oddly.
 - **Error mapping:** surface `causes` from 422 responses back to your own API consumers in a shape your app already uses for validation errors, rather than passing CarryBee's raw shape through.
 - **Store `consignment_id`** on your own Order record — it's the key for tracking, cancelling, reverse pickup, and exchange calls.
-

10. Environment variables template

CarryBee — pick environment

CARRYBEE_ENV=sandbox # sandbox | production

Sandbox

CARRYBEE_SANDBOX_BASE_URL=https://sandbox.carrybee.com

CARRYBEE_SANDBOX_CLIENT_ID=

CARRYBEE_SANDBOX_CLIENT_SECRET=

CARRYBEE_SANDBOX_CLIENT_CONTEXT=

Production

CARRYBEE_PRODUCTION_BASE_URL=https://developers.carrybee.com

CARRYBEE_PRODUCTION_CLIENT_ID=

CARRYBEE_PRODUCTION_CLIENT_SECRET=

CARRYBEE_PRODUCTION_CLIENT_CONTEXT=

Webhook

CARRYBEE_WEBHOOK_INTEGRATION_SECRET= # value to echo back in
X-CB-Webhook-Integration-Header

CARRYBEE_WEBHOOK_SIGNING_SECRET= # used to validate
X-Carrybee-Webhook-Signature

Fill these in from the merchant dashboard → **API Credentials / Webhook Integration** pages. Don't commit real values — this file only documents the shape.

11. Open questions to confirm in Sandbox before going live

- Exact hashing/signing algorithm behind **X-Carrybee-Webhook-Signature** (HMAC-SHA256 of raw body? shared secret only?) — not spelled out on the docs page, worth a quick test-event round trip to confirm before writing verification code.
- Exact payload shape for **order.paid**, **order.returned-at-sorting**, **order.returned-in-transit**, **order.returned-to-merchant** — documented as base-envelope-only, but worth confirming with a live test event since a couple of neighboring "returned" events do carry **reason/remarks**.
- Retry policy / timeout for webhook delivery (how many attempts, backoff) — not documented; treat your endpoint as needing to be reliably fast and idempotent regardless.
- Response body shapes for **GET** list/detail endpoints other than "list cities" and "create order" (list zones/areas, list stores, get order details) — the docs page didn't show a worked example for each; the enclosing envelope will match §5, but call each once in Sandbox to capture the exact **data** shape for your DTOs.